

A3-PS023-TC465

Minimal 2-node Cloud Hypervisor VM Controller

Vishal Parmar
vishal.parmar@iitg.ac.in

Hardik Shah
hardik.shah@iitg.ac.in

Problem Description

- Edge and small-scale deployments need VM management, but enterprise hypervisor stacks depend on multiple daemons and heavy components. These cost more memory, CPU and operational complexity than such environments can spare.
- Cloud Hypervisor is a minimal VMM built on Linux KVM, but it has real limits such as it runs one process per VM and exposes only a local, per-VM API.
- No way to manage VMs across multiple hosts.
- No way to decide where a new VM should run (scheduling and placement).
- No way to restrict who can control the VMs (access control).

Background

- **Heavy stacks**: Enterprise hypervisor stacks runseveral daemons. Edge sites cannot spare that memory, CPU and operational effort.
- **Local-only API:** Cloud Hypervisor runs one process per VM and exposes only a per-VM API on a local socket
- **No control plane:** Nothing manages VMs across hosts, decides where a new VM runs, or restricts who may control it.

Literature & RFC Review

[1]KVM: the Linux Virtual Machine Monitor (Kivity et al., Ottawa Linux Symposium, 2007): Describes how KVM turns the Linux kernel into a hypervisor by exposing hardware virtualization through /dev/kvm, with a userspace process handling each VM's devices. Relevance: Cloud Hypervisor runs on this model, so it explains why our design uses one hypervisor process per VM on KVM-capable hosts.

[2]RFC 6750- The OAuth 2.0 Authorization Framework- Bearer Token Usage (Jones and Hardt, IETF, 2012): Defines how a bearer token is sent in the HTTP Authorization header, how servers reject invalid tokens with 401, and the security requirements, such as using TLS and keeping tokens out of URLs. Relevance: We follow its conventions for authenticating API requests, without deploying a full OAuth 2.0 server.

Proposed Solution

We Propose a control plane made of a controller and two worker agents, running on each host.

• **Controller**: Exposes authenticated API endpoints through which a user can create, start, stop and list VM. Keeps a registry of VMs, checks worker health, and chooses the host for each new VM.

• **Worker Agent**: Each worker agent runs locally on its host and translates the controller requests into calls to cloud hypervisor management API.

• **Resource-Based-Placement**: Each worker reports allocatable memory (total, minus a host reserve, minus memory already committed to its VMs). The controller picks the healthy worker with the most free memory that fits the request, and rejects the request if neither fits.

• **Authentication**: Client → controller: a bearer API key.
Controller → worker: a different bearer token or mutual TLS, per worker.

Optimization Proposed

- **Reserve-aware memory accounting:** Host reserve and committed memory are tracked, so a worker is never overcommitted.
- **Cached health and capacity:** Workers report via heartbeats, so placement reads SQLite instead of calling hosts live.
- **Reconcile on restart:** A restarted worker rescans its VM sockets so the registry matches reality.
- **Lean runtime:** Async FastAPI handlers, SQLite in WAL mode and few dependencies keep overhead low.

Solution Architecture & Design

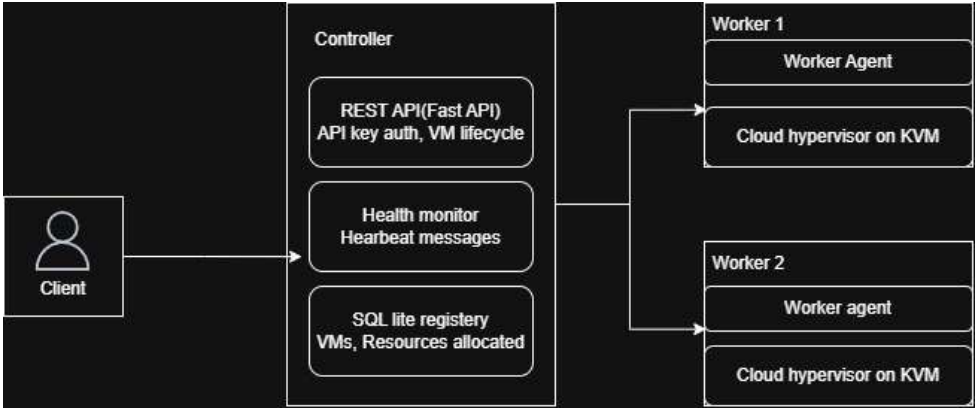


Figure: System Architecture UML Diagram

Data Flow Diagram.

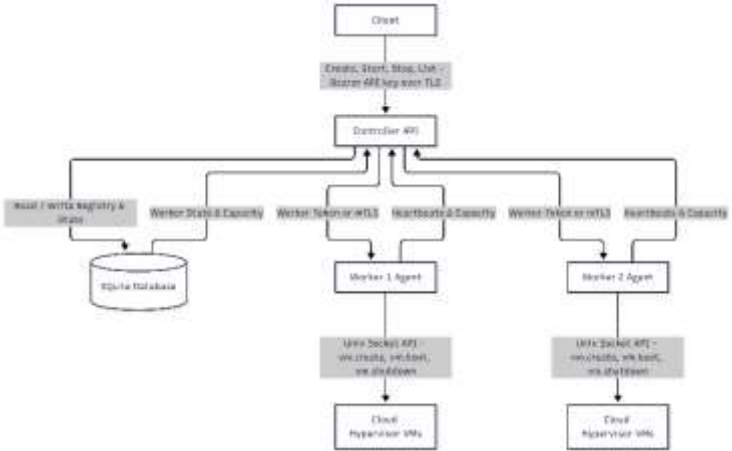


Figure: Data Flow Diagram (DFD)

Deployment Timeline



Bibliography I

1 https://www.kernel.org/doc/ols/2007/ols2007v1-pages-225-230.pdf

2 https://www.rfc-editor.org/info/rfc6750/

Thank You for Your Attention!